

TDV 2215 Terminal Interrupt System Analysis & Debug Guide

Interrupt Service Routine Overview

The TDV 2215 terminal uses a **Z80 processor** with **Interrupt Mode 1** and **NMI** support. The system implements 6 distinct interrupt sources with specific handlers and priorities.

Z80 Interrupt Vector Table (Memory Layout)

Standard Z80 Interrupt Vectors

Address	Vector	ISR Address	Function Name	Purpose
0x0000	RST 0	0x0000	RST0	Power-on/Reset entry point
0x0021	-	0x0295	host_serial_interrupt_handler	Z80SIO Channel A (Host)
0x0029	-	0x0295	host_serial_interrupt_handler	Z80SIO Channel A (Host)
0x002D	-	0x0443	keyboard_interrupt_handler	Z80SIO Channel B (Keyboard)
0x003D	-	0x052C	vsync_interrupt_handler	Display VSYNC timing
0x0406	NMI	0x0406	power_fail_interrupt_handler	Power failure detection
0x043D	RST 6.5	0x043D	ISR_RST_6_5	Unused (returns immediately)
0x048E	-	0x048E	printer_interrupt_handler	Printer interface

Detailed Interrupt Service Routine Analysis

1. RST 0 - System Reset (0x0000)

```
c
void RST0(void) {
    disableMaskableInterrupts();
    terminal_boot_sequence();
    return;
}
```

Purpose: Power-on reset and system initialization entry point **Trigger:** CPU reset, power-on **Action:** Disable interrupts and start boot sequence **Emulator Requirements:**

- Must be first code executed at power-on
- PC starts at 0x0000

2. Host Serial Interrupt Handler (0x0295)

c

```
void host_serial_interrupt_handler(void) {  
    interrupt_activity_flag = 0x5e;  
    return;  
}
```

Purpose: Handle data from host computer via Z80SIO Channel A **Trigger:** Z80SIO Channel A receive/transmit ready **Memory:** Sets `interrupt_activity_flag` (0x5E21) = 0x5E **Action:** Minimal ISR - just signals activity to main loop **Emulator Requirements:**

- Z80SIO Channel A implementation
- IRQ triggered on RX/TX ready
- Vectored interrupt to 0x0295

3. Keyboard Interrupt Handler (0x0443)

c

```
void keyboard_interrupt_handler(void) {  
    interrupt_activity_flag = 0x5e;  
    return;  
}
```

Purpose: Handle keyboard input via Z80SIO Channel B **Trigger:** Z80SIO Channel B receive ready (key pressed) **Memory:** Sets `interrupt_activity_flag` (0x5E21) = 0x5E **Action:** Minimal ISR - signals activity to main loop **Emulator Requirements:**

- Z80SIO Channel B implementation
- IRQ on keyboard data available
- Vectored interrupt to 0x0443

4. VSYNC Interrupt Handler (0x052C) - CRITICAL

c

```
undefined1 vsync_interrupt_handler(void) {
    set_to_1_by_vsync = 1;                // Mark VSYNC occurred

    // Read control port 2 and check status
    bVar3 = (in_io_00000060 & 0xc) >> 2;
    if ((expected_control_port2_value_maybe & 3) != bVar3) {
        expected_control_port2_value_maybe = expected_control_port2_value_maybe & 0x28 | bVar3;
        jump_to___5c22__can_be_0x29e_return_From_isr();
    }

    // Handle cursor blink timing
    display_refresh_control = (display_refresh_control & 3) << 1;

    // Process timeout counters
    if (interrupt_activity_flag == 0) {
        if ((BYTE_ram_5f0d == 0) && (--DAT_ram_5e22 == 0)) {
            bVar1 = bVar1 | 8; // Set timeout bit
        }
    } else {
        DAT_ram_5e22 = 30000; // Reset timeout counter
    }

    // Update status port
    value_for_statusport_2 = value_for_statusport_2 & bVar2 | bVar1;
    if (statusport2_flag == 0) {
        write_to_io_port_0x60(value_for_statusport_2);
    }

    // Clear interrupt activity flag
    interrupt_activity_flag = 0;

    // Process timer channels (0-3)
    // Handle printer status check
    // Handle keyboard data processing

    return status;
}
```

Purpose: Display timing, cursor blink, timeout management, timer processing **Trigger:** Video display VSYNC signal (50/60Hz) **Critical Functions:**

- Display refresh synchronization
- Cursor blink timing (every other VSYNC)

- 30-second timeout counter (30000 VSYNCs $\approx$ 30 seconds at 60Hz)
- Multi-channel timer management
- I/O port status updates **Memory Variables:**
- `set_to_1_by_vsync`: VSYNC occurrence flag
- `display_refresh_control` (0x5DF9): Display timing control
- `interrupt_activity_flag` (0x5E21): Activity from other interrupts
- `timer_channel_flags` (0x5E25): Multi-timer control bits
- `DAT_ram_5e22`: 30-second timeout counter **Emulator Requirements:**
- **60Hz or 50Hz periodic interrupt** (most critical for terminal operation)
- I/O port 0x60 read/write capability
- Timer countdown functionality

5. Power Fail Interrupt Handler (0x0406) - NMI

c

```
undefined1 power_fail_interrupt_handler(void) {
    // Check if system is in operational state (0x8080)
    if (system_status_word == 0x8080) {
        if (BYTE_ram_5e20 != 0) {
            BYTE_ram_5e20 = 2; // Set power fail state
            return;
        }
        read_keyboard_data(); // Emergency keyboard read
    }

    // Check interrupt mask and re-enable if needed
    bVar2 = readInterruptMask();
    if ((bVar2 & 8) != 0) {
        enableMaskableInterrupts();
    }
    return;
}
```

Purpose: Handle power failure detection and graceful shutdown **Trigger:** NMI (Non-Maskable Interrupt) from power monitoring circuit **Action:** Emergency state save, keyboard data preservation **Emulator Requirements:**

- NMI capability
- Power monitoring simulation (optional)

6. Printer Interrupt Handler (0x048E)

c

```
void printer_interrupt_handler(void) {  
    return; // Currently unused  
}
```

Purpose: Handle printer interface interrupts **Trigger:** Printer ready/error status **Action:** Currently no-op (feature not implemented) **Emulator Requirements:** Optional (returns immediately)

7. RST 6.5 Handler (0x043D)

c

```
void ISR_RST_6_5(void) {  
    return; // Unused  
}
```

Purpose: Unused interrupt vector **Action:** Immediate return **Emulator Requirements:** None (just return)

Interrupt Initialization & Configuration

Interrupt Mask Setup (during `initialize_system_variables`)

c

```
disableMaskableInterrupts();  
setInterruptMask(10); // Enable specific interrupt sources
```

Interrupt Mask Value: `10` (0x0A binary = 00001010)

- Bit 1: Enable (likely Z80SIO interrupts)
- Bit 3: Enable (likely VSYNC interrupts)

Interrupt Vector Setup

The system initializes several interrupt-related pointers:

c

```
_ISR_RETURN_ADDRESS = 0x10;  
ISR_65_RETURN_ADDRESS = 0x10;  
ISR_VSCYN_RETURN_ADDRESS = 0x10;
```

Function Pointer Table (Interrupt Dispatch)

Multiple function pointers point to `0x29e` initially:

c

```
pointer_to_0x29e_1 = 0x29e;
pointer_to_0x29e_2 = 0x29e;
// ... (8 total pointers)
```

These are likely updated during operation for dynamic interrupt handling.

Emulator Implementation Requirements

Critical Interrupt Timing

- 1. **VSYNC (Most Important):** 50-60Hz periodic interrupt
 - **Address:** 0x052C (vsync_interrupt_handler)
 - **Frequency:** 16.67ms intervals (60Hz) or 20ms (50Hz)
 - **Critical for:** Display timing, cursor blink, timeouts
- 2. **Z80SIO Interrupts:** As needed for I/O
 - **Host Serial:** 0x0295 when data received/transmitted
 - **Keyboard:** 0x0443 when key pressed

Memory Locations Your Emulator Must Support

Address	Name	Purpose	Access
0x5E21	interrupt_activity_flag	Interrupt coordination	R/W
0x5DF9	display_refresh_control	Display timing	R/W
0x5E25	timer_channel_flags	Timer control	R/W
0x5E22	DAT_ram_5e22	30-second timeout	R/W
0x5E20	Power fail state	Emergency state	R/W
0x0060	Control Port 2	I/O status	R/W

I/O Port Requirements

- **Port 0x60:** Control/Status port (critical for VSYNC handler)
- **Z80SIO Ports:** Channel A (host) and B (keyboard)

Debug Breakpoints for Interrupt Testing

1. Interrupt Enable Check

Breakpoint: After initialize_system_variables (0x08C9) **Test:**

- Interrupts should be enabled

- Interrupt mask should be set to 10 (0x0A)
- Check if `setInterruptMask(10)` was called

2. VSYNC Interrupt Entry

Breakpoint: 0x052C (vsync_interrupt_handler entry) **Test:**

- Should be called every 16.67ms (60Hz) or 20ms (50Hz)
- `set_to_1_by_vsync` should be set to 1
- Check I/O port 0x60 is readable

3. VSYNC Interrupt Processing

Breakpoint: 0x0634 (end of VSYNC handler) **Test:**

- `interrupt_activity_flag` should be cleared to 0
- Timer counters should be decremented
- Status port should be updated

4. Serial Interrupt Testing

Breakpoint: 0x0295 (host_serial_interrupt_handler) **Test:**

- Should be called when Z80SIO has data
- `interrupt_activity_flag` set to 0x5E
- Check Z80SIO status registers

Breakpoint: 0x0443 (keyboard_interrupt_handler) **Test:**

- Should be called on keyboard input
- `interrupt_activity_flag` set to 0x5E

5. Interrupt Activity Processing

Breakpoint: In main loop where `interrupt_activity_flag` is checked **Test:**

- Flag should be set by interrupt handlers
- Should be cleared by main processing loop
- Check proper interrupt/main loop coordination

Common Emulator Interrupt Issues

1. Missing VSYNC Interrupt

Symptoms: Terminal appears frozen, no cursor blink, timeouts don't work **Fix:** Implement periodic interrupt every 16.67ms calling 0x052C

2. Incorrect Interrupt Vectors

Symptoms: System crash on interrupt, random behavior **Fix:** Ensure interrupt vectors point to correct ISR addresses

3. Z80SIO Not Interrupting

Symptoms: No keyboard input, no host communication **Fix:** Implement Z80SIO interrupt generation on data ready

4. Missing I/O Port 0x60

Symptoms: VSYNC handler crashes or behaves incorrectly **Fix:** Implement I/O port 0x60 read/write

5. Interrupt Mask Problems

Symptoms: Too many or too few interrupts **Fix:** Check `setInterruptMask(10)` implementation

Test Sequence for Interrupt Validation

Phase 1: Basic Interrupt Setup

1. Set breakpoint at 0x08C9 (after interrupt init)
2. Verify interrupt mask = 10
3. Verify interrupts are enabled

Phase 2: VSYNC Interrupt Testing

1. Set breakpoint at 0x052C
2. Run for 100ms, should hit 5-6 times (60Hz)
3. Check `interrupt_activity_flag` behavior

Phase 3: Serial Interrupt Testing

1. Simulate Z80SIO data ready
2. Verify interrupt to 0x0295 or 0x0443
3. Check flag setting

Phase 4: Integration Testing

1. Run full system with all interrupts
2. Monitor flag coordination
3. Verify no interrupt conflicts

This comprehensive interrupt analysis should help you implement and debug the interrupt system in your TDV 2215 emulator.